JAVA using BlueJ

Flow Control Statement

The statements inside your source files are generally executed from top to bottom, in the order that they appear. *Control flow statements*, however, break up the flow of execution by employing decision making, looping, and branching, enabling your program to *conditionally* execute particular blocks of code.

1. Conditional Statement
2. Looping statement
3. Jumping Statement

Conditional/Selection statement

- **Decision making statement** is depending on the condition block need to be executed or not which is decided by condition. In java programming language there are three types of conditional statement:

- If statement
- If else statement
  - Nested if else statement
  - Else if ladder statement
- Switch case statement

if Statement

- This is the simplest form of branching statement. When we have only one condition to evaluate, we should use this statement. It performs a course of action if the condition evaluates to true, otherwise, it ignores the action.
- **Syntax:**

```
If(conditional expression)
{
    Statement 1
    Statement 2
    ..........
    Statement n
}
```

Lab Assignment

- Write a program to check whether the entered number is positive or not.
- Write a program to input an integer number and print its absolute value.
- Write a program input any char from user and convert it in upper case if it is in lower case.

if-else statement

The if else control structure is used where either of two action are to be performed depending upon the result of a conditional expression. It contains two block of statement. If the conditional expression evaluates to true, the statement(s) in if block are executed and if the result is false, then statement(s) in the else block get executed.

Syntax:

```
if(conditional expression)
{
        statement 1
        statement 2
        ……….
}
else
{
        statement 1
        statement 2
        ……….
}
```

Lab Assignment

- Write a program to input an integer number and check number is **EVEN** or **ODD.**
- Write a program to input any integer number and check number is **BUZZ Number** or Not. (A number that ends with 7 or divisible by 7, is called buzz number).
- Write a program to display whether the entered character is Uppercase or Lowercase.
- Write a program to check whether the entered number is divisible by 5 or Not.
- Write a program input cost price, selling price and check whether is profit or loss and also print profit/loss amount.
- Write a program input any one digit integer number and its square contains as last digit or not. Ex: 5=2**5 ,** 6=3**6**

IF ELSE IF LADDER

- The if-else-if ladder statement executes one condition from multiple statements. The execution starts from top and checked for each if condition. The statement of if block will be executed which evaluates to be true. If none of the if condition evaluates to be true then the last else block is evaluated.

```
if(condition_expression_One)
{
   statement1;
}
else if (condition_expression_Two)
{
   statement2;
}
else if (condition_expression_Three)
{
   statement3;
}
else
{
   statement4;
}
```

Lab Assignment

- Write a program to input serial number and print the corresponding day of week.
- Write a program input 5 subject marks and find total, percentage and grade based on given condition.

  Grade A – per >=80
  Grade B – per >=70 and <80
  Grade C – Per >=60 and <70
  Grade D – per>=50 and <60
  per<50 is FAIL

- Create a program to calculate the amount that a customer pays for the taxi that he hires based on the following conditions.

KMS Travelled	Amount per KM(Rs)
First 10 Kms	30
Next 20 Kms	20
Next 40 Kms	15
Above 70 Kms	12

Nested IF ELSE

- A nested if is a statement that has another if or else in its body.
- Hear, the inner if block condition executes only when outer if block condition is true.
- Syntax:

```
If(condition)
{
        If(condition)
        {
                Statement
        }
}
else
{
        Statement

}
```

Lab Assignment

- Program that will accept three numbers from the user , and find the largest number.

Switch Statement

A switch statement is used when there is requirement to check multiple condition in a program. It provides an easy way to branch to different parts of the code in program based on the value of an expression. It is a substitute for the large series of else if statements.

Syntax:

```
switch(expression)
{
        case  constant:
                statement;
        break;
        case  constant:
                statement;
        break;
        ..
        ..
        default:
                statement;
}
```

Looping statement

A for loop is used to repeat a block of statement enclosed in curly braces. It contains three optional expressions enclosed in parentheses and separated by semicolons, followed by statements executed in the loop.

- For loop
- While loop
- Do while loop

For Loop

for loop is a statement which allows code to be repeatedly executed. For loop contains 3 parts Initialization, Condition and Increment or Decrements.

Syntax:

```
for(initial value ;condition; increment/decrement)
{
        statement 1
        statement 1
        ..............
        statement n
}
```

Variation in FOR Loop

- A for loop can have multiple variations that makes more flexible and efficient.
  - Multiple initialization and update expression
  - Optional expression
  - Infinite loop
  - Empty loop

Multiple initialization and update expression

- A for loop may contain multiple initialization and update expressions. The multiple expressions must be separated by commas.

Example:

```
int i,j;

for(i=1,j=20; I,=10; i++,j--)
{
        System.out.println(i+" "+j)
}
```

Multiple Initialization

Multiple update expression

Optional Expression

- By now you are familiar that a for loop contains three types of expressions: initialization expression(condition) and an update expression. But, all these expressions are optional, i.e., we can skip any or all of these expressions depending upon the requirement of the program.

Case 1:

```
Int i=2;
for( ;i<=20;i=i+2)
{
   System.out.println(i);
}
```

Case 2:

```
Int i=2;
for( ;i<=20; )
{
   System.out.println(i);
   i=i+2;
}
```

Case 3:

```
Int i=2;
for( ; ; )
{
  System.out.println(i);
   i=i+2;
   if(i>20)
   {
            break;
   }
}
```

Infinite Loop

- A for loop will execute its statement block infinite number of times if we skip the condition. If the test expression (condition) is not present in the loop, it will never return false. Hence, it will execute endlessly and become an infinite loop.

Case 1

```
int I
for(i=1; ;i++)
{
    System.out.pritln(i);
}
```

Case 2

```
int i;
for( ; ; )
{
  System.out.pritln(i);
}
```

Case 3

```
int i;
for(i=1;i<=i;i++)
{
  System.out.pritln(i);
}
```

Here , i<=I will always be true.

Empty Loop

- A loop without any statement in its body is called an empty loop

```
int i
for(i=2;i<=50;i++)
{
}
```

Or

```
for(i=2;i<=50;i++);
```

Lab assignment

1. Create a program to create a table of given number.
2. Write a program to print cube of numbers 1 to 5.
3. Create a program to print the factorial of a number. Factorial is 1*2*3…*n.
4. Write a program to accept a number from the user and display its divisors.
   Ex: 12= 1,2,3,4,6,12
5. Write a program which accept a number (n) and find the sum from 1 to n.
6. Write a program to input any 10 integers number and find largest number.
7. Create a program that will display the elements of Fibonacci series between 1 and 100. eg. 0,1,1,2,3,,5,8,13,21,34,55,89

While loop

The while loop is entry controlled loop. The while loop keeps on executing the block of statement till the specified test condition evaluates to true. When the test condition becomes false, the program control is transferred out of the loop and passes to the statement after the body of the loop.

Syntax:

```
while(condition)
{
        Statement 1;
        Statement 2;
        ……..
        Statement n;
}
```

Lab Assignment

1. Write a program that will find the **reverse** of a given number.
2. Write a program which accept a number from user and **count the number of digits**.
3. Write a program that will find the **Sum of digits** of a given Number.
4. Write a program which accept a integer number from user and check, number is **Palindrome** or Not. Eg 121 ,444,100001 etc.
5. Write a program which accept a 3 digit integer number and check number is **Armstrong** or not. Ex 153, 370,371,407.
6. Write a program input any integer number and finds its largest and smallest digit. Ex. 92584 the min=2 and max=9
7. Write a Program which accept a number from user and check whether it is a **Disarium Number** or not. **Ex.** 135 is a DISARIUM ($1^1+3^2+5^3 = 135$), some other **DISARIUM** are 89, 175, 518 etc.)

do-while

The do while loop is an **exit controlled loop**, since the body of loop is executed first and then the test condition is evaluated. If it evaluates to false, then the loop is terminated, otherwise it is repeated. This means do while loop always executes **at least once**, even if the condition is evaluated to false.

Syntax:

```
do
{
        statement 1;
        statement 1;
        ..............
        statement 1;
}while(condition);
```

Lab Assignment

- Write a program which print sum of negative numbers, sum of positive odd numbers and positive even numbers from a list of numbered entered by the user. The list terminates when the user enters a Zero.
- Write a program which accept integers number from user until the sum of all given number is less than 100.
-

Jumping Statement

The jump statement are used to transfer the program control unconditionally to some other location. Java provides three types of jump statements:

- Break statement
- Continue statement
- Return statement

Break Statements

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the end of any immediate loop or switch body.
- To do all this we have to specify a break jumping statements whenever we want to terminate from the loop.

- **Syntax:** break;

- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

  NOTE: As break jumping statements ends/terminate loop of one level . so it is refered to use return or goto jumping statements , during more deeply nested loops.

Continue Statements.

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the beginning of any immediate loop.
- To do all this we have to specify a continue jumping statements whenever we want to terminate any particular condition and restart/continue our execution.

Syntax: continue;

- **NOTE:** This jumping statements always used with the looping structure while, do while, for loop .

Return Statement

- return statement is used to transfer program's control from called function to calling function, it's secondary task is to carry value from called function to calling function.
- If function's return type is void, you can use return only to return program's control to the calling function, and if there is a return type you will have to return same type of value to the calling function.

Syntax:

```
return;
```

Or

```
return (value)
```